Cheffy AI Dining Concierge

Complete Implementation Report

Desi Dhamaka Restaurant Group · Everything built from day one · July 13, 2026

This PDF is the definitive inventory of Cheffy — the AI dining concierge on the Desi Dhamaka website. It covers the floating mascot and chat UI, Gemini streaming backend, CMS knowledge layer, tool calling, personality and session memory, Semantic RAG Knowledge Base, context orchestrator, admin CMS modules, outlet isolation, ChefGaa live-menu redirects, Netlify functions, Supabase migrations, and deploy requirements.

1. Executive Summary

- Product name: Cheffy — AI Restaurant Assistant (Powered by ChefGaa)
- Live public site: https://desi-dhamaka-admin.netlify.app
- Admin AI Concierge: /admin/integrations/ai-concierge
- Admin Knowledge Base: /admin/integrations/knowledge-base
- Stack: React + TypeScript + Vite · Netlify Functions · Gemini LLM · Supabase (CMS + pgvector RAG)
- Outlets: South Plainfield · Oak Tree (Edison) · Lawrenceville — never mixed in one reply
- Menu policy: Cheffy does not invent dishes or prices; guests are sent to the outlet ChefGaa live menu

2. End-to-End Architecture

Guest opens Cheffy !’ React UI collects the message !’ CMS knowledge loads for the selected outlet !’ Context Orchestrator optionally retrieves semantic RAG chunks !’ enriched request hits Netlify ai-concierge !’ Gemini streams the reply !’ UI parses action tokens, presentation cards, and follow-up chips !’ session memory persists in browser sessionStorage. If the LLM is unavailable, a deterministic CMS responder answers from live website data.

```
Customer !’ Semantic Search (optional) !’ CMS + RAG context !’ Gemini !’ Cheffy UI
```

3. Implementation Timeline (from the start)

Phase	What was delivered
Foundation	Floating mascot, chat shell, AIProvider on public routes
Gemini gateway	Netlify ai-concierge, streaming SSE, system prompt
CMS knowledge	Outlet-aware builder, tools, deterministic fallback
Personality	Session prefs, greetings, chips, hospitality voice
Admin Concierge	AI settings, personality, prompts, logs, sandbox
Semantic RAG	Upload, chunk, embed, pgvector search, playground
Orchestrator	CMS + RAG + tools + business rules before each LLM call
Menu handoff	Demo menu removed; all menu CTAs !’ ChefGaa per outlet

4. Frontend — Guest Experience (LIVE)

Entry & shell

- src/App.tsx — mounts AIProvider + Cheffy on all public location routes
- src/components/ai/CheffyContext.tsx — conversation engine, orchestrator, streaming, history
- src/components/ai/Cheffy.tsx — floating mascot, bubbles, open/close chat, inactivity nudge
- Asset: public/cheffy/cheffy-mascot.png

Mascot, motion & emotion

- CheffyMascotState — phases: hidden !’ peek !’ greeting !’ idle !’ listening !’ thinking !’ speaking !’ celebrating
- CheffyAnimator — GSAP transform-only motion; respects prefers-reduced-motion
- First-paint CSS hide + useLayoutEffect so mascot never flashes before intro
- Location-specific welcome copy for each NJ outlet
- Emotion bridge: intent detection !’ mascot thinking/speaking states

Chat UI

- ChatWindow, ChatHome, MessageBubble, MessageStreamRenderer, Typing / DynamicTyping indicators
- Quick actions, input chips, follow-up chips after replies

- Presentation cards: Offer, Location, Contact, Recommendation (View Live Menu)

Action system

- Parses tokens: [BUTTON:], [ORDER:], [ONLINE ORDERING:], [MENU:], [CALL:], [MAP:], [ACTION:switch_location:]
- Menu / Order external URLs open the outlet ChefGaa page in the same tab
- Phone / email / maps / reservation / switch-location handled safely in CheffyActionBar

5. AI Core — Providers, Prompt, Tools, Personality

Providers

- Default LIVE provider: Gemini (gemini-2.0-flash) via Netlify — keys never in the browser
- Optional: OpenAI and Claude paths on the server when env keys exist
- Mock provider for local/dev fallback
- Client always calls /.netlify/functions/ai-concierge (JSON or SSE stream)

System prompt (canonical)

- src/config/ai/systemPrompt.ts — single source of truth for client + Netlify
- Rules: restaurant-only scope, anti-hallucination, outlet isolation, hospitality voice
- Menu rule: direct guests to ChefGaa live menu; never invent dishes or prices
- Action-token format instructions for buttons and deep links

Intents detected

menu · vegetarian · offers · reservation · catering · order · recommend · hours · contact · location · greeting · faq · kids · party · buffet · gallery · unknown

Registered tools (LIVE against CMS)

- getRestaurantInfo, getHomepageContent, getOffers, getGallery, getReviews, getSEO
- getCurrentLocation, navigateToPage
- Future (not live): Menu tool, Reservation tool, ChefGaa cart — feature-flagged only

Personality & session memory

- Guest prefs: dietary, spice, mood, dining purpose, family/kids — stored in sessionStorage
- Time-of-day greetings, nudge copy, hospitality closings
- Dynamic chips adapt to last topic and preferences
- Conversation history + conversation UUID for correlation/logging

6. CMS Knowledge Layer (LIVE)

- buildCMSKnowledge(locationId) aggregates homepage, restaurant settings, offers, gallery, reviews, SEO
- navigation.menu resolves to the outlet ChefGaa order URL from Restaurant Settings
- navigation.order points to /{location}/online-ordering on the website
- Deterministic CMS responder answers when LLM is unavailable
- unavailableMenu() explains live menu is on online ordering + View Live Menu button

7. Semantic RAG Knowledge Base

Cheffy can learn from uploaded documents — not only CMS tables. Documents are extracted, chunked, embedded with Gemini text-embedding-004 (768 dimensions), stored in Supabase pgvector, and retrieved before Gemini answers when semantic RAG is enabled.

Deliverables status

Deliverable	Status
AI Knowledge Base Module	Done — admin + RAG services
PDF / DOCX / TXT Upload	Done (+ MD, HTML, CSV)
Categories & Metadata	Done
Version Control	Done — semantic_document_versions
Document Preview	Partial — text/chunk preview
Smart Text Extraction	Done — pdf-parse / mammoth
Semantic Chunking	Done — paragraph + overlap
Embedding Generation	Done — Netlify index fn
Supabase pgvector	Done — migration 033

Semantic Search	Done
Similarity Scoring	Done — cosine
Search Playground	Done — admin UI
Re-indexing	Done
Document Lifecycle	Done — upload/version/delete

Knowledge categories

Category ID	Label
restaurant_policies	Restaurant Policies
faqs	FAQs
catering	Catering
private_parties	Private Parties
events	Events
festival_info	Festival Information
brand_story	Brand Story / About
press_releases	Press Releases
awards	Awards
training	Training (private / internal)
future_menu	Future Menu Documents

Key files

- Admin UI: src/admin/pages/KnowledgeBasePage.tsx
- RAG services: src/services/rag/* (categories, chunker, cache, retriever, repository)
- Functions: semantic-knowledge-index.mts, semantic-knowledge-search.mts
- Libs: semanticEmbeddings.ts, semanticTextExtractor.ts, semanticSupabase.ts
- Migration: supabase/migrations/033_semantic_knowledge.sql
- Storage bucket: semantic-knowledge (private, admin-only, 50MB)

8. Context Orchestrator

- enrichAIRequestWithOrchestrator wraps CMS enrichment — does not replace core AI architecture
- Plans sources: outlet · business rules · session · personality · CMS · tools · ± semantic RAG
- Trims RAG chunks (max ~4 chunks / ~1200 tokens) before injecting as semanticKnowledge tool result
- Wired from CheffyContext on every guest message

`src/services/ai/orchestrator/{contextOrchestrator,sourcePlanner,businessRules}.ts`

9. Admin CMS Modules

AI Concierge — /admin/integrations/ai-concierge

- Sections: General, Personality, Knowledge, Providers, Prompt, Conversation, Suggestions
- Also: Analytics, Testing/Sandbox, Logs, Error Logs, Advanced, location overrides
- Roles: super_admin / manager / staff (staff read-only)
- Tables from migration 032_ai_management.sql

Knowledge Base — /admin/integrations/knowledge-base

- Upload with title, description, category, outlet scope, public/private visibility
- Document list with index status badges (pending / processing / indexed / failed / stale)
- Re-index, delete, version history, chunk preview, search playground

10. Outlet / Location Awareness

- Uses LocationContext selectedLocationId for every knowledge build and RAG query
- Prompt rule: never mix South Plainfield, Oak Tree, and Lawrenceville
- Mascot welcomes are location-specific
- Switch-location action can move the guest between outlets
- RAG documents can be global (location_id null) or outlet-scoped
- Order URLs come from Restaurant Settings order_url per outlet (validated, never mixed)

11. Menu & Online Ordering Integration

The on-site demo menu was removed. Menu is no longer a separate website page. Every Menu entry point (header, footer, experience cards, signature carousel, Cheffy buttons) redirects to the selected outlet's ChefGaa online ordering URL from CMS Restaurant Settings. If no outlet is selected, the guest is sent to the location gate.

- Legacy /{location}/menu !' MenuRedirectPage !' ChefGaa (or / if no location)
- Cheffy menu intents !' explain live menu + [BUTTON:View Live Menu|orderUrl]
- Admin knowledge flags keep menu/chefgaa tools OFF by design (link strategy)

12. Netlify Functions

Function	Purpose
ai-concierge.mts	LLM gateway — Gemini/OpenAI/Claude + SSE
semantic-knowledge-index.mts	Extract !' chunk !' embed !' store
semantic-knowledge-search.mts	Query embed !' match_semantic_chunks

13. Supabase Migrations & Schema

032_ai_management.sql

- ai_settings, ai_personality, ai_provider_settings, ai_prompt_versions
- ai_suggested_questions, ai_followups, ai_feature_flags
- ai_conversation_logs, ai_tool_logs, ai_error_logs, ai_audit_log
- users.ai_access_level + admin-only RLS

033_semantic_knowledge.sql

- Extension: vector (pgvector)
- Tables: semantic_documents, semantic_document_versions, semantic_chunks, semantic_index_jobs
- RPC: match_semantic_chunks — outlet-aware, public-only, category-filtered cosine search
- Index: IVFFlat on embedding vector(768)
- Storage: semantic-knowledge bucket

14. Environment Variables

Variable	Where	Purpose
VITE_AI_PROVIDER	Client	Provider name (gemini)
AI_PROVIDER	Netlify	Server provider
GEMINI_API_KEY	Netlify only	LLM + embeddings
GEMINI_MODEL	Netlify	Default gemini-2.0-flash
GEMINI_* tuning	Netlify	Temp, top_p, tokens, streaming
CHEFFY_PROMPT_VERSION	Netlify	Prompt version tag
OPENAI_* / ANTHROPIC_*	Netlify	Optional providers
VITE_SUPABASE_*	Client	CMS + admin
SUPABASE_SERVICE_ROLE_KEY	Netlify only	RAG index/search

Never expose LLM API keys or the service role key to the browser.

15. Live vs Admin-Only

Capability	Guest site	Admin
Floating mascot + chat	Yes	—
Gemini streaming	Yes	Sandbox too
CMS tools + outlet context	Yes	Toggles
Menu !' ChefGaa	Yes	—
Session chips / memory	Yes	Edit seeds

Semantic RAG retrieval	Yes if indexed	Upload/index
AI settings / logs	—	Yes
Provider keys / embeddings	Server only	Config UI

16. Deploy Checklist

- 1. Apply migrations 032 and 033 to Supabase (pgvector must be available)
- 2. Set Netlify env: GEMINI_API_KEY, Supabase URL/anon, SUPABASE_SERVICE_ROLE_KEY
- 3. Confirm netlify.toml functions directory and external modules (pdf-parse, mammoth)
- 4. Optional: enable ai_feature_flags.semantic_rag; upload docs and reindex
- 5. Deploy with npm run deploy (or Netlify CI)
- 6. Verify Cheffy on a location page and Knowledge Base / AI Concierge in admin

17. Intentional Non-Features / Gaps

- No live dish-level menu catalog inside Cheffy — ChefGaa is the source of truth
- Offers RAG category is not dedicated; offers primarily come from CMS offers module
- Document preview is extracted text/chunks, not an embedded PDF viewer
- Claude client path is mock; server handlers exist when keys are set
- Training category is private and excluded from public match_semantic_chunks
- Admin AI settings persist in Supabase; runtime LLM is primarily env-driven on Netlify

18. Primary Code Map

Guest UI

```
src/components/ai/*  ·  src/App.tsx
```

AI services

```
src/services/ai/*  ·  src/config/ai/*  ·  src/services/cms/knowledge/*
```

RAG

```
src/services/rag/*  ·  src/types/semanticKnowledge.ts
```

Orchestrator

```
src/services/ai/orchestrator/*
```

Admin

```
src/admin/pages/AIConciergePage.tsx  ·  KnowledgeBasePage.tsx
src/services/aiAdmin/*  ·  src/admin/hooks/use*Admin.ts
```

Server

```
netlify/functions/ai-concierge.mts
netlify/functions/semantic-knowledge-*.mts
netlify/functions/lib/{cheffySystemPrompt,aiConfig,semantic*}.ts
```

Database

```
supabase/migrations/032_ai_management.sql
supabase/migrations/033_semantic_knowledge.sql
```

19. Outcome

Cheffy is a production dining concierge: warm animated host on the public site, outlet-safe answers from live CMS data, optional document intelligence via Semantic RAG, Gemini streaming through a secure Netlify gateway, and full admin control for personality, prompts, knowledge documents, and testing. Guests who ask for the menu are guided to the correct ChefGaa online ordering experience for their selected location — never to invented dishes or a mixed-outlet URL.